

Vibe Coding을 위한 개발도구 (Codex)



1. 도구 소개

❖IDE 기반 도구

- AI 전용 IDE로 제작된 개발 도구
- 대표적인 예
 - VSCode + AI 관련 Extension(무료)
 - 사용 가능한 AI 기능 : Github Copilot, Claude, Gemini 등
 - Cursor, Windsurf, Antigravity : 유료 구독 방식
 - VSCode 기반 : VSCode의 코어 에디터와 UI를 활용
 - » VSCode의 Extension을 모두 지원하는 것은 아님

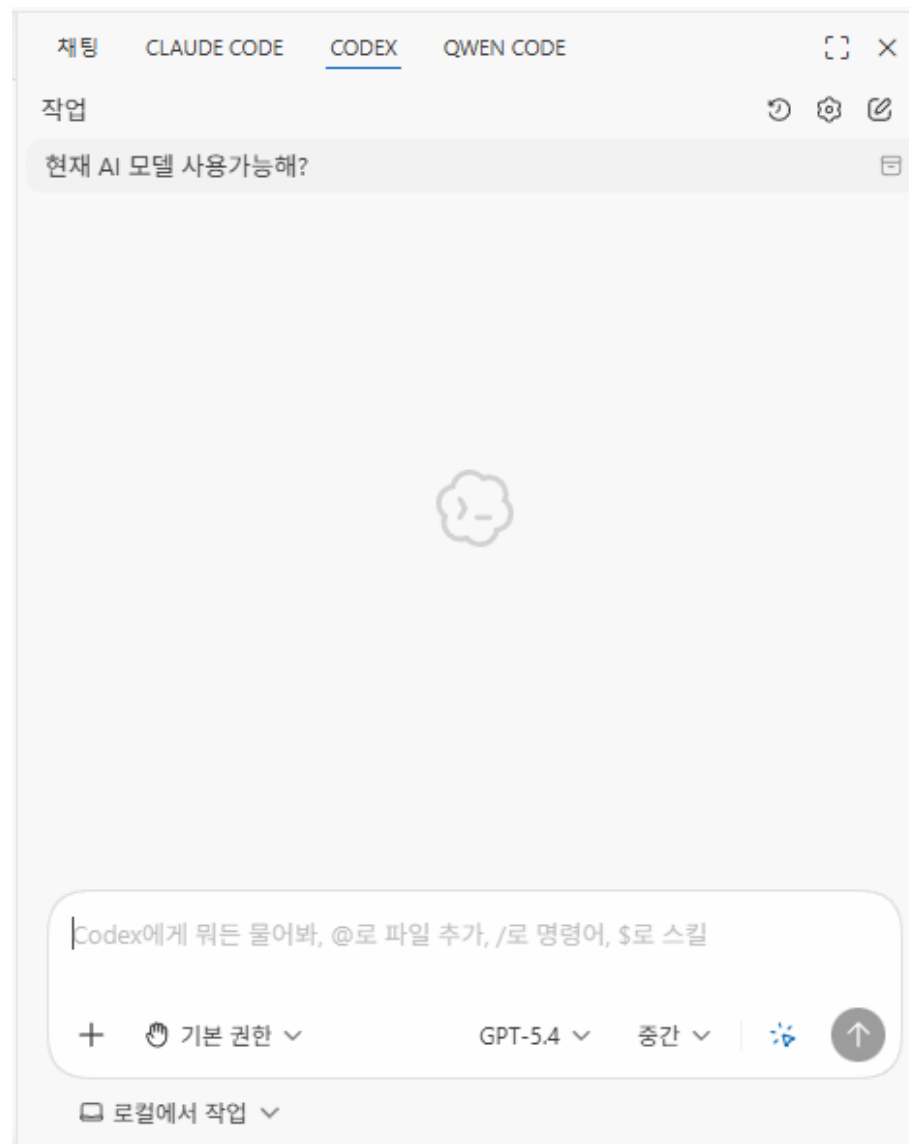
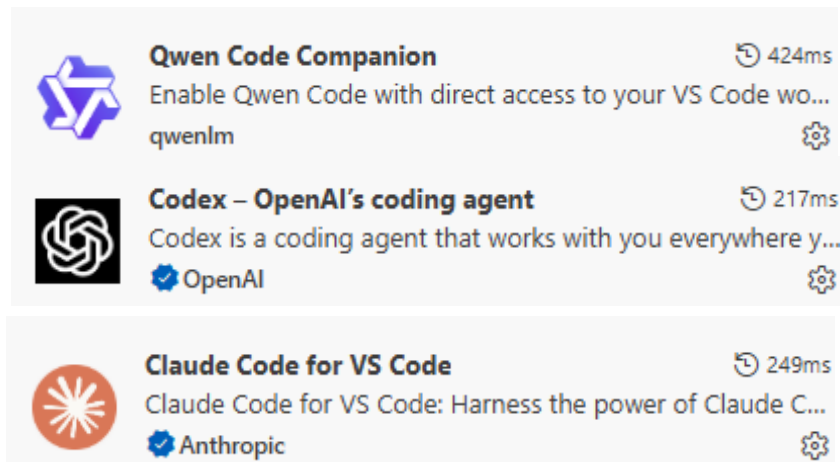
❖CLI 기반 도구

- 터미널에서 명령어를 이용하도록 만들어진 개발 도구
 - 터미널에서 실행하므로 어느 도구에도 연동이 가능함
- 대표적인 예
 - Claude Code : Claude
 - Antigravity CLI : Google Gemini를 이용한 CLI 도구
 - Codex : Open AI Chat GPT
 - OpenCode : 여러 AI 모델을 연동할 수 있는 CLI 개발 도구

2. VSCode 기반 AI 코딩 도구

❖ VSCode 확장 프로그램 기반의 도구

- Codex
- Claude Code
- Antigravity - 별도의 IDE 도구 사용
- Qwen
- Cursor



3. Codex 개요

❖Codex : OpenAI가 제공하는 AI 기반 소프트웨어 개발 에이전트

- 두가지 도구

- Codex : 독립실행형으로 실행하는 AI 코딩 에이전트 도구
- Codex CLI : TUI(Terminal User Interface) 에서 사용하는 코딩 전용 도구

- 둘 중 Codex CLI를 권장하는 이유

- 코딩만을 하는 것이 아님
 - 코드만을 개발하는 것이라면 Codex 앱을 사용해도 되지만 개발한 앱을 구성하고 서버에 배포하고 하려면 다양한 터미널 명령어를 사용하게 되므로 처음부터 TUI 환경에 적응하는 것이 바람직함
 - 다양한 도구를 프로젝트 단위로 직접 설정하고 사용하기 위함
- TUI 환경에 익숙해지면 향후 바이브 코딩에서 가장 많이 이용되는 Claude Code에 쉽게 접근할 수 있음.
- 저사양의 컴퓨터에서도 개발이 가능하다.
 - Codex 앱은 상당히 무거운 편이지만 Codex CLI는 터미널을 실행할 수 있는 수준이면 개발이 가능함.
- AI 에이전트가 작업하는 흐름을 쉽게 볼 수 있음
 - AI가 실행하는 과정, 계획을 수립하는 과정을 터미널 환경에서 모두 볼 수 있음

❖Codex CLI 도구 테스트를 위한 예제 프로젝트 준비

- `git clone https://github.com/stepanowon/test-svc`

3.1 Codex CLI

❖Codex CLI 설치

- 이미 환경설정에서 설치하였음
 - 윈도우 : powershell -ExecutionPolicy Bypass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
 - mac : curl -fsSL https://chatgpt.com/codex/install.sh | sh

❖환경 설정

- statusline 설정 : codex 실행 후 /statusline
 - model-with-reasoning, current-dir, git-branch, branch-changes
 - approval-mode, context-used, five-hour-limit
 - 더 필요하다면 추가할 수 있음
- ponytail, caveman 플러그인 설치
 - Plus 구독을 할 때 토큰을 절약하기 위한 도구
 - Ponytail : 과도한 코드 생성, 오버엔지니어링을 막아주어 토큰 절약해줌
 - codex plugin marketplace add DietrichGebert/ponytail
 - codex plugin add ponytail@ponytail
 - caveman : 응답 출력을 간략하게 하여 출력 토큰 비용을 절감해줌. caveman, caveman-help
 - npx skills add JuliusBrussee/caveman -a codex

3.1 Codex CLI

❖codex CLI 실행 명령어

- codex [options] <command> [prompts]
- commands
 - plugin
 - agents : 현재 실행중인 모든 에이전트 세션 조회
 - mcp : 외부 MCP 서버 관리
 - update : 최신 버전으로 업데이트
 - doctor : 설정, 인증, 실행 상태 진단
- options
 - --local-provider : ollama 등을 이용한 로컬 LLM 서버로 연결할 때 로컬 제공자를 지정(lmstudio, ollama)
 - --yolo 또는 --dangerously-bypass-approvals-and-sandbox
 - 모든 확인 프롬프트를 건너뛰고 샌드박스 없이 명령을 실행함. 위험하기 때문에 샌드박스 처리된 격리된 환경에서만 실행할 것을 권장함.
 - --ask-for-approval <APPROVAL_POLICY>
 - 모델이 명령을 실행하기 전에 인간의 승인을 요구하는 시점을 설정함. on-request, never 중의 한 값을 지정.
 - on-request : 모델이 언제 사용자에게 승인을 요청할지 스스로 판단
 - never : 절대 사용자 승인을 요청하지 않음. 실행 실패는 즉시 모델에게 반환됨

3.2 세션과 컨텍스트

❖ 세션(Session)

- Codex CLI를 실행해서 종료할 때까지의 하나의 작업 단위
- 세션 안에서 사용자의 지시, Codex의 응답, 파일 읽기/쓰기 기록, 명령어 실행 결과 등 모든 상호작용이 누적됨
- 연관된 작업을 이어나갈 때는 기존 세션을 이어가는 것이 효율적임

❖ 컨텍스트(Context)

- Codex가 현재 시점에 참조할 수 있는 모든 정보를 저장하는 제한된 용량의 저장 공간
 - 읽은 파일 내용, 입력한 모든 프롬프트, 응답 결과 등
 - 세션이 시작할 때 자동으로 로드된 메모리(시스템 프롬프트, AGENTS.md, MCP 도구, Skill 정보)
 - 이 밖에도 터미널 화면에는 보이지 않지만 처리되고 있는 많은 정보들
- 컨텍스트 공간이 부족해지면 Compaction 해야 함
 - /compact 명령어를 이용한 명시적 compaction, 일정 크기에 도달하면 자동으로 compaction
 - 기존 컨텍스트 내용을 요약해 저장하고 컨텍스트 공간 확보
 - 대화 내용, 응답 중 중요한 내용이 있다면 반드시 파일 형태로 저장해야 함

3.2 세션과 컨텍스트

❖ Session, Context, Compact 을 비유하자면...

■ Session

- 하나의 작업 책상을 펴놓은 연속된 시간
- 책상위에는 AI와의 대화 내용, 읽은 파일들, 실행한 명령어와 결과 등이 쌓여 나감
- 동일한 프로젝트가 진행될 때는 새로운 세션을 만들지 말고 계속 이어가는 것이 바람직함
 - 새로운 세션을 만들거나 /clear하면 대화내용, 읽은 파일 정보가 모두 삭제됨
 - 그래서 중요한 정보는 채팅에 의존하지 말고 반드시 파일로 저장해야 함
- 새로운 프로젝트가 진행되거나 전혀 다른 작업을 진행할 때 새로운 세션을 시작

■ Context

- 작업을 수행하는 책상의 크기
- 작업을 계속해서 수행하면 대화, 파일 내용, 응답결과 등이 계속해서 책상에 쌓여나감 -> 컨텍스트 사용량 증가
- OpenAI의 모델의 경우 컨텍스트 용량 : 1.05M 토큰

■ Compact

- 책상 위의 정보들을 요약해서 책상 한켠에 보관하여 책상의 공간을 확보함
- 요약되는 과정 중에 일부 정보가 유실될 수 있음
- 따라서 중요한 정보는 markdown 파일로 원본 형태로 보관해야 함

3.3 슬래시 명령어 치트 시트

❖주요 슬래시 명령어1 : 세션/작업 관리

- /new : 같은 CLI 세션 안에서 새 대화 시작
- /clear : 터미널 화면 + 대화를 함께 초기화하고 새 대화(세션) 시작 (Ctrl+L은 화면만 지움)
- /resume : 저장된 세션 목록에서 골라 이어서 진행
- /compact : 긴 대화를 요약해 컨텍스트 확보
- /fork : 현재 대화(세션)를 새 ID로 복제하여 진행(기존 세션은 저장됨)
- /side (/btw) : 메인 대화(세션)를 건드리지 않는 임시 사이드 대화 시작
- /quit, /exit CLI 종료

❖주요 슬래시 명령어2 : 모델/스타일 지정

- /model : 사용할 모델(및 reasoning effort) 선택
- /fast : 모델의 Fast 서비스 티어 켜기/끄기
- /plan : 계획(Plan) 모드로 전환(Shift+Tab 으로 수행해도 됨)
- /goal : 지속형 목표 설정/조회 (edit, pause, resume, clear 서브옵션)

3.3 슬래시 명령어 치트 시트

❖주요 슬래시 명령어3 : 권한/샌드박스

- /permissions : 승인 정책 프리셋 변경 (Auto, Read Only 등)
- /approve : 자동 리뷰가 거부한 최근 작업을 한 번 재시도 승인
- /sandbox-add-read-dir <경로> : 특정 디렉토리에 읽기 권한 추가

❖주요 슬래시 명령어4: 터미널 커스터마이징/유틸리티

- /copy : 최근 완료된 응답 복사 (Ctrl+O와 동일)
- /statusline : 하단 상태줄 항목 구성
- /theme : 코드 문법 강조 테마 선택
- /logout : 로그아웃. 로그인했던 자격증명 제거
- /export : 현재 세션의 대화 내용을 markdown 으로 내보내기
- /memories : 대화 내용을 메모리에 자동 저장할 지, 사용하지 않을지를 지정
 - 저장된 메모리는 다음 세션에서 로드되도록 할 수 있음

3.3 슬래시 명령어 치트 시트

❖주요 슬래시 명령어5 : 세션정보/진단/터미널작업

- /status : 현재 모델, 승인 정책, 쓰기 가능 경로, 토큰 사용량 확인
- /usage : 계정 토큰 사용량 조회 (daily/weekly/cumulative)
- /ps : 백그라운드 터미널 목록 확인
- /stop : 모든 백그라운드 터미널 중지

❖주요 슬래시 명령어6 : 확장 기능

- /mcp : 연결된 MCP 서버/도구 목록 조회
- /plugins : 설치된/탐색 가능한 플러그인 조회
- /hooks : Life Cycle Hook 조회 및 활성화/비활성화
- /agents, /subagents
 - Codex가 작업중에 서브에이전트를 Spawn 했을 때 main agent와 sub agent 사이를 이동하며 작업 내용을 들여다보고 싶을 때 이용하는 명령어
- /skills : 스킬 탐색 및 정보 조회

3.3 슬래시 명령어 치트 시트

❖주요 슬래시 명령어7 : 코드/파일

- /diff : Git diff(변경사항) 확인
- /review : 작업 트리에 대한 코드 리뷰 요청
- /init : 현재 코드베이스를 분석한 후 AGENTS.md 생성
- /ide : 열려있는 IDE 파일/선택 영역을 컨텍스트로 포함할 것인지 여부를 on/off 토글

❖Context 명령어

- 파일을 선택하여 지정할 때 : @
- 직접 스킬 사용을 지정할 때 : \$
- 터미널 명령어를 직접 실행할 때 : !

3.4 주요 슬래시 명령어

❖ /init

- 현재 작업 디렉토리의 구조와 파일을 분석해서 기본 메모리 지침(AGENTS.md)를 생성함
- AGENTS.md
 - AI 모델이 이 프로젝트를 이해하기 위한 핵심 컨텍스트가 됨
 - 매번 챗을 시작할 때 자동으로 읽어서 프로젝트의 맥락으로 사용함
- 생성된 AGENTS.md 예시

저장소 가이드라인

프로젝트 구조 및 모듈 구성

- `src/index.js`에서 Express 앱과 미들웨어를 설정하고 `PORT`(기본값 `3000`)에서 수신합니다.
- `src/routes.js`는 HTTP 라우트를 정의하고 요청 데이터를 DAO 호출로 연결합니다.
- `src/tododao.js`에는 LokiJS 기반 Todo 작업이 있습니다. 데이터베이스는 메모리 방식이며 서버를 시작할 때마다 샘플 데이터가 다시 생성됩니다.
- `views/index.ejs`는 서버 렌더링 훅 페이지이고, 정적 브라우저 자산은 `public/`에 둡니다.
- `build/`는 Babel 생성 결과입니다. `build/`가 아닌 `src/`를 수정하세요. 현재 메모리 구성에서는 `sample.db`를 사용하지 않습니다.

빌드, 테스트 및 개발 명령어

- `npm install`: `package-lock.json` 기준으로 의존성을 설치합니다.
- `npm run dev`: Nodemon으로 `babel-node src/index.js`를 실행하며, 소스 변경 시 다시 로드합니다.
- `npm run build`: Babel로 `src/`를 `build/`로 변환합니다.
- `npm start`: 빌드 후 `node build/index.js`를 실행합니다.

.....(생략)

3.4 주요 슬래시 명령어

❖수준별 AGENTS.md

- ~/.codex/AGENTS.md
 - 사용자 수준의 메모리
 - 로컬 머신의 모든 프로젝트에 적용되는 전역 규칙
- 프로젝트 수준의 메모리 지침
 - /init에 의해 만들어진 AGENTS.md
 - 그대로 쓰기 보다는 직접 다듬고 불필요한 내용을 삭제하는 등의 작업이 권장됨
 - 프로젝트 수준에서 사용되고 git과 같은 도구를 이용해 소스 제어되는 메모리
 - 다른 개발자들과 공유
- 하위 디렉토리의 메모리 지침
 - 작업 디렉토리 내의 하위 디렉토리의 AGENTS.md
- 병합 순서

```
<git-root>/AGENTS.md (또는 .override.md)
└─ <중간 디렉토리>/AGENTS.md
    └─ <현재 작업 디렉토리>/AGENTS.md
```



3.4 주요 슬래시 명령어

- AGENTS.md에 항상 포함시키면 좋을 것
 - 사용 언어 : 사용언어를 지정하지 않으면 기본적으로 영어우선으로 답변함
 - "오버엔지니어링 금지"
 - 이 지침을 지정하지 않으면 AI가 불필요하게 복잡한 코드. 당장 필요하지 않은 코드까지 미리 생성해버림
 - 중요 작업 규칙과 작업 전 체크리스트
 - 프로젝트 개요 및 아키텍처
 - 어떤 서비스인지, 주요 디렉토리와 각각의 역할
 - 사용중인 기술 스택
 - 빌드, 테스트, 실행 명령어
 - 코딩 컨벤션
 - DO NOT
 - 예시 : 승인없이 수정하면 안되는 파일
 - 보안/민감 정보 처리 방법
 - 예시) 자격증명, API KEY 등은 절대 하드코딩하지 말것
- AGENTS.md 에는 꼭 필요한 규칙만 포함시킬 것
 - "당신만 모르지 AI는 이미 알고 있다" --> 널리 알려진 기술, 용어에 대한 장황한 설명 금지

3.4 주요 슬래시 명령어

❖ /usage [args]

- args : daily, weekly, cumulative
 - 사용량을 일별, 주별, 누적으로 보여줌
 - 색이 연할수록 사용량이 적었다는 것
- args 없이 실행하면 Redeem usage limit reset 기능 사용
 - 5H 리셋 시간까지 기다리지 않고 사용량 한도를 즉시 풀어주는 기능
 - 이용하려면 크레딧을 구매해야 함. 현 시점 일주일에 한번 리셋할 수 있는 크레딧이 주어짐



3.4 주요 슬래시 명령어

❖ /ps, /stop

- 다음 명령어를 순차적으로 실행하여 결과를 확인
 - "개발모드로 백엔드 서비스를 실행시켜줘"
 - /ps : 백그라운드로 실행중인 서비스 확인

/ps

Background terminals

- npm run dev

↳

할일 목록 서비스가 3000번 포트에서 시작되었습니다!

GET /

1 background terminal running · /ps to view · /stop to close

- /stop : 백그라운드 실행 프로세스 모두 중지

3.4 주요 슬래시 명령어

❖ /new, /resume

- /new : 새로운 세션 시작. 기존 세션의 대화를 이어받지 않음
- /resume : 이전 세션으로 전환 재개, 이전 컨텍스트 복원
- 다음 명령 실행
 - /new

```
>_ OpenAI Codex (v0.153.4)
```

```
model:      gpt-5.6-terra medium  /model to change  
directory:  D:\dev\workspace\vibe\test-svc  
permissions: YOLO mode
```

Tip: New Use /fast to enable our fastest inference with increased plan usage.

```
> Ask Codex to do anything
```

```
gpt-5.6-terra medium · D:\dev\workspace\vibe\test-svc · main · No changes · never · Context 0% used
```

- /resume 명령 실행 하여 중단된 세션으로 전환

3.4 주요 슬래시 명령어

❖ /permissions

- 4가지 옵션 중에서 선택

옵션	파일 읽기	파일 수정	명령 실행	인터넷 접근	승인 방식
1. Read Only	가능	승인 필요	제한적/승인 필요	승인 필요	매우 보수적
2. Ask for approval	가능	가능	가능	승인 필요	일반적인 기본값
3. Approve for me	가능	가능	가능	가능하지만 위험 작업만 질문	자동화 수준 높음
4. Full Access	가능	가능	가능	가능	거의 묻지 않음

❖ /compact

- 현재 세션의 대화와 작업 기록을 요약해서 컨텍스트 사용량을 줄임.
- context 사용량이 지나치게 높을 때(70%이상)라면 직접 해주는 편이 바람직함

3.4 주요 슬래시 명령어

❖ /status : 현재 모델, 승인 정책, 쓰기 가능 경로, 토큰 사용량 확인

- 특히 plus 구독이라면 5h limit의 초기화 시간을 확인

/status

> OpenAI Codex (v0.153.4)

Visit <https://chatgpt.com/codex/settings/usage> for up-to-date information on rate limits and credits

```
Model: gpt-5.6-terra (reasoning medium, summaries auto)
Directory: D:\dev\workspace\vibe\test-svc
Permissions: Full Access
Agents.md: AGENTS.md
Account: stepanowon@gmail.com (Plus)
Thread name: 전체 코드에 수행
Collaboration mode: Default
Session: 01a07463-9bb8-7df0-b9d8-f059a397a103
```

```
5h limit:      [██████████░░░░░░░░░░] 92% left (resets 13:16)
Weekly limit:  [██████████░░░░░░░░░░] 82% left (resets 06:30 on 8 Sep)
```

3.4 주요 슬래시 명령어

❖ /review : 코드 리뷰 실행

- 기존 설정은 git diff 정보를 이용해서 코드 리뷰 진행함
- 만일 전체 코드를 리뷰하고 싶다면 프롬프트 추가
 - "/review 전체 코드에 대해서 코드 리뷰를 수행해"

<< Code review finished >>

- 생성된 빌드 결과가 프로젝트의 ES 모듈 설정과 호환되지 않아 기본 시작 명령이 실패합니다.

Full review comments:

- [P1] ES 모듈 형식을 유지해 서비스를 시작 가능하게 하세요 - D:\dev\workspace\vibe\test-svc\build\index.js:3-3
npm start는 빌드 후 이 파일을 실행하지만, 프로젝트가 "type": "module"인 상태에서 생성물이 require()를 사용합니다. 따라서 Node 22 이상에서 ReferenceError: require is not defined in ES module scope로 즉시 종료되어 서버가 시작되지 않습니다. Babel이 ES 모듈을 보존하도록 설정하거나 생성물과 실행 방식을 CommonJS로 일치시켜야 합니다. AGENTS.md:20-21
- [P2] 사용하지 않는 Playwright 런타임 의존성을 제거하세요 - D:\dev\workspace\vibe\test-svc\package.json:27-27
현재 저장소에는 playwright를 import하거나 실행하는 코드가 없으므로, 프로덕션 설치마다 사용되지 않는 브라우저 자동화 패키지와 전이 의존성만 설치됩니다. 설치 시간·배포 크기·공급망 관리 범위를 불필요하게 늘리므로 실제 기능을 추가할 때까지 제거하거나 테스트 도구라면 개발 의존성으로 옮겨야 합니다. AGENTS.md:6

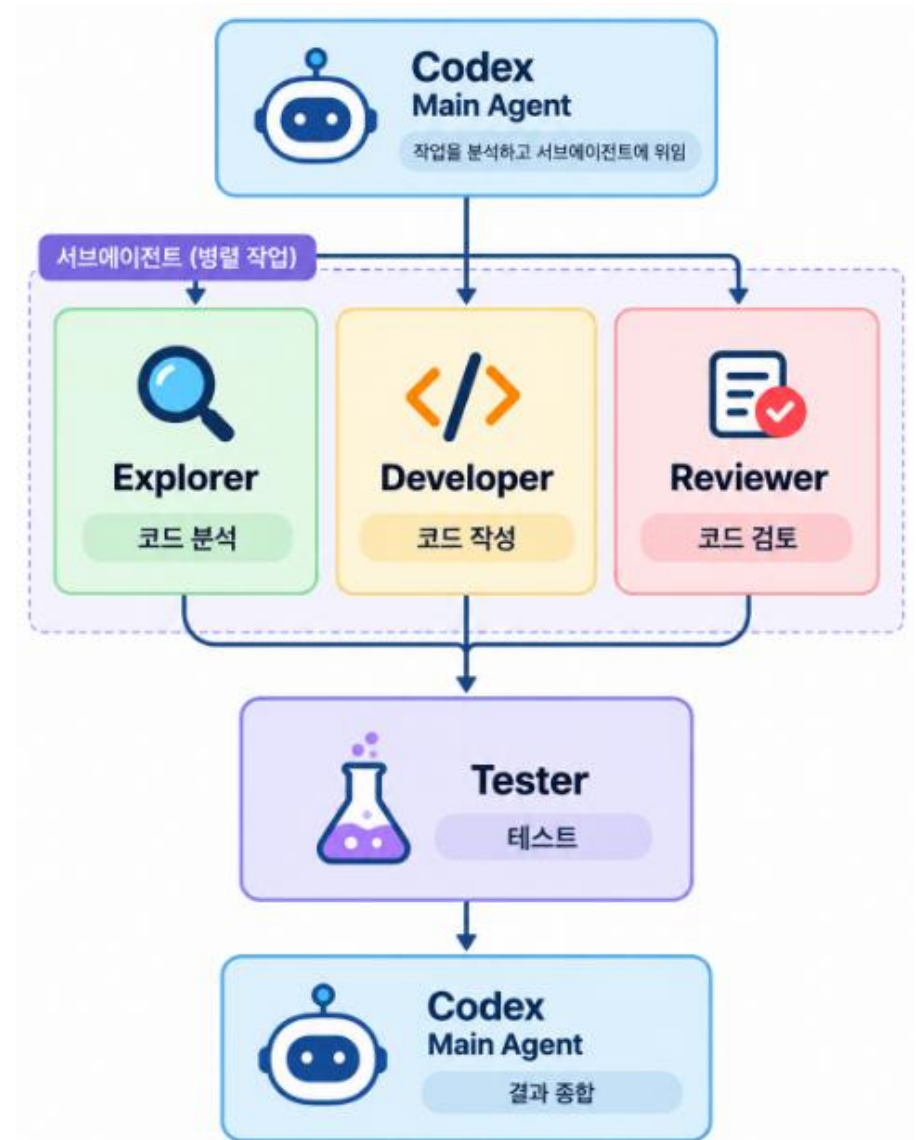
4. 서브에이전트

❖ 서브에이전트(SubAgent)란?

- Main Agent가 특정 작업을 독립적인 작업 컨텍스트에서 수행할 수 있도록 위임하기 위해 사용하는 별도의 에이전트

❖ 서브에이전트 특징

- 독립된 컨텍스트 사용
 - 메인 에이전트의 전체 대화 컨텍스트에 모든 작업을 쌓지 않고, 별도의 작업 스레드에서 처리
- 역할 전문화
 - explorer, reviewer, tester처럼 역할별로 나눌 수 있으며, 직접 사용자 정의 에이전트를 정의할 수 있음
- 메인 에이전트가 작업을 위임하고 결과를 종합
 - 메인 에이전트는 오케스트레이터 역할만을 수행하며 서브에이전트의 작업이 메인에이전트의 컨텍스트를 오염시키지 않음
- 병렬 실행 : 동시에 여러 개 서브에이전트가 실행



4. 서브에이전트

❖비유하자면...

- 팀장(Main Agent)이 임시 팀원(SubAgent)에게 작업을 위임함
- 팀원은 독립적인 책상(Context 1M 토큰)을 가지고 작업함
- 작업이 완료된 후 팀장에게 결과를 보고하고 팀원 사라짐
- 여러 팀원에게 맡기면 작업을 병렬로 수행할 수 있음
- 팀장은 임시 팀원으로부터 결과만 돌려받으므로 팀장의 책상은 지저분해지지 않음
 - MainAgent의 컨텍스트는 결과물 만을 돌려받기 때문에 Context 사용량이 조금만 증가함

❖built-in agent

- default : 범용 일반 작업 수행
- worker : 코드 구현, 수정, 디버깅
- explorer : 코드베이스의 파일 읽기, 탐색, 실행 흐름 파악

❖서브에이전트는 AI에 의해 자동 선택되지만 프롬프트를 입력할 때 직접 지정할 수 있음

4. 서브에이전트

❖ 사용자 정의 서브에이전트

- 사용자 수준 : ~/.codex/agents/*.toml
- 프로젝트 수준 : .codex/agents/*.toml

❖ 서브에이전트 전역 설정

- .codex/config.toml

```
# enabled : 서브에이전트(멀티에이전트) 기능 활성화/비활성화
# default_subagent_model : 서브에이전트에 적용되는 기본 모델. 개별 에이전트에서 모델을 직접 지정할 수 있음
# default_subagent_reasoning_effort : 기본 적용할 추론 강도
# max_concurrent_threads_per_session : 한 세션에서 동시에 실행가능한 서브에이전트 스레드의 최대 갯수
# interrupt_message : 서브에이전트의 turn이 중간에 중단된 경우, 그 사실을 모델의 컨텍스트에 메시지로 남길지 지정
```

```
[agents]
enabled = true
max_concurrent_threads_per_session=6
default_subagent_model="gpt-5.6-luna"
default_subagent_reasoning_effort="medium"
interrupt_message=true
```

4. 서브에이전트

❖ 서브에이전트 예시1

▪ .codex/agents/backend-architect.toml

```
name = "backend-architect"
description = """
백엔드 API, 서비스 경계, 데이터 모델을 설계·검토한다.
확장성, 성능, 운영 복잡성의 균형이 필요한 작업에 사용한다.
"""

model = "gpt-5.6-terra"
model_reasoning_effort = "high"
developer_instructions = """
요구사항과 기존 시스템을 먼저 확인하고, 구현 가능한 최소 설계를 제안한다.

- API 계약, 서비스 경계, 데이터 모델, 보안과 오류 처리를 함께 검토한다.
- 분산 구조, 캐시, 인덱스, 샤딩은 현재 요구사항의 근거가 있을 때만 제안한다.
- 일관성, 성능, 확장성, 운영 복잡성의 핵심 트레이드오프를 짧게 설명한다.
- 필요한 경우에만 엔드포인트 예시, 스키마, 다이어그램, 병목 지점을 제공한다.
- 과도한 마이크로서비스 분할과 조기 최적화는 피한다.
"""
```

4. 서브에이전트

❖서브에이전트 예시2

▪ .codex/agents/docs-architect.toml

```
name = "docs-architect"
```

```
description = """
```

코드베이스를 분석해 정확하고 구조화된 기술 문서를 작성한다.

아키텍처, 데이터 흐름, 주요 모듈과 통합 지점을 설명할 때 사용한다.

```
"""
```

```
model = "gpt-5.6-luna"
```

```
model_reasoning_effort = "medium"
```

```
developer_instructions = """
```

실제 코드와 설정을 근거로 기술 문서를 작성한다. 확인되지 않은 내용은 추측하지 말고 불확실성을 표시한다.

- 시스템 목적, 구조, 주요 모듈, 데이터 흐름, API·통합 지점을 설명한다.
- 중요 설계 결정은 코드에서 근거를 찾을 수 있을 때만 이유와 함께 설명한다.
- 문서는 개요에서 구현 세부 사항 순으로 구성하고, 독자 수준에 맞춰 명확하게 쓴다.
- 표, 코드 예제, Mermaid 다이어그램은 이해에 실질적으로 도움이 될 때만 사용한다.
- 요청 범위에 필요한 섹션만 포함하며 반복으로 분량을 늘리지 않는다.

```
"""
```

4. 서브에이전트

❖ 서브에이전트 사용

- Codex CLI를 중단후 재시작
- "docs-architect 서브에이전트를 이용해 코드베이스에 대한 문서화를 수행후 README.md에 저장하고 backend-architect 서브에이전트를 이용해 소스 코드에 대한 분석을 수행 후 개선점을 찾아 수정해줘"

```
> docs-architect 서브에이전트를 이용해 코드베이스에 대한 문서화를 수행후 README.md에 저장하고 backend-architect 서브에이전트를 이용해 소스 코드에 대한 분석을 수행 후 개선점을 찾아 수정해줘
```

↳ Hook · PONYTAIL:FULL

- 요청대로 문서화와 백엔드 분석을 병렬로 맡기고, 저는 변경 범위와 검증을 정리하겠습니다. 간결한 수정 원칙을 적용하기 위해 **Ponytail** 지침도 함께 따르겠습니다.

- Explored

- ↳ Read SKILL.md (ponytail:ponytail skill)

- Started `/root/documentation`

- Started `/root/backend\_analysis`

- 실행중 /subagents 명령어 실행

Subagents

Select an agent to watch. alt + ← previous, alt + → next.

- ```
> 1. • Main [default] (current) 01a074b4-482d-7042-bf53-bb14520a265b
 2. • /root/documentation 01a074b4-7efe-7f93-be06-c5a57cf19fb2
 3. • /root/backend_analysis 01a074b4-9789-7d01-a331-25201851d06c
```

Press enter to confirm or esc to go back

## 4. 서브에이전트

### ❖ 서브에이전트를 작성할 때 developer\_instruction에 포함할 권장 사항

- 필수 요구 사항
- 접근 방식
- 사용할 도구(MCP)
- DO, DO NOT
- 서브에이전트에는 수행할 작업에 대한 지나치게 상세한 작업을 포함시키지 않는 것이 바람직함
  - 독립된 컨텍스트(1M 토큰)을 할당하는 것에 중점을 둠
  - 상세한 작업 지침은 Skill에 의해서 수행되도록 하는 것이 바람직함 --> 그래야 재사용성이 높음

### ❖ 서브에이전트 활용 방안

- 나만의 서브에이전트를 만들 수도 있지만 이미 검증된 제3자의 것을 활용하는 것도 바람직함
- 예시 : github star 수가 높은 것들
  - <https://github.com/VoltAgent/awesome-codex-subagents>
  - <https://github.com/VKirill/codex-starter-kit>

# 5. MCP

## ❖MCP

- AI 모델이 외부 도구, 서비스와 안전하고 구조적으로 상호작용할 수 있도록 만든 표준 프로토콜
- 모델이 플러그인처럼 다양한 도구를 호출하여 사용할 수 있게 함.

## ❖MCP 사용 예시

- 데이터베이스 연결 MCP
  - SQL 문이 아닌 자연어로 데이터베이스 관리 작업 수행
  - "살고 있는 지역(region)이 '서울'인 사용자를 조회해줘"
- 클라우드 리소스 관리 MCP
  - AWS, Kubernetes와 같은 클라우드 리소스 관리 기능 제공
  - "TestServer 라는 이름의 EC2 인스턴스 생성해줘"
- Git, Github 리소스 관리
  - Git, Github 관련 작업을 자연어로 수행
  - "현재 브랜치를 main 브랜치로 merge 해줘"
- CI/CD 자동화
  - Jenkins MCP 등을 이용해 빌드/배포 상태 조회. 특정 빌드 잡 실행

## 5. MCP

### ❖애플리케이션 개발시 자주 사용하는 MCP 서버

- Context7
  - 사용하는 라이브러리의 최신 버전 설명서와 코드예제를 참조하도록 하는 MCP
  - <https://github.com/upstash/context7>
- Postgresql
  - Postgresql 데이터베이스에 접근할 수 있는 기능 제공
  - 스키마 관리, 사용자/권한 관리, 쿼리 실행, 함수/트리거 관리
  - <https://github.com/HenkDz/postgresql-mcp-server>
  - <https://smithery.ai/server/@HenkDz/postgresql-mcp-server>
- Supabase
  - Supabase 프로젝트에 연결하는 기능을 제공하는 MCP
  - Supabase : 백엔드 기능을 제공하는 SaaS 서비스. DB, 인증 등의 기능을 제공함
  - <https://github.com/supabase-community/supabase-mcp>
- markdown mcp
  - PDF, Office, HTML, 이미지 등을 markdown 형식으로 변환해주는 MCP 서버
  - <https://github.com/microsoft/markitdown/tree/main/packages/markitdown-mcp>

## 5. MCP

### ❖애플리케이션 개발시 자주 사용하는 MCP 서버(이어서)

- Playwright MCP
  - 브라우저 자동화 기능을 제공하는 MCP 서버. 여러 브라우저 지원하지만 mcp 설정을 별도로 해야 함
  - LLM을 이용해 브라우저상의 웹페이지와 상호작용할 수 있음
  - <https://github.com/microsoft/playwright-mcp>
- Chrome Devtools MCP
  - Chrome 브라우저를 제어하고 검사하는 MCP 서버. Chrome 브라우저만 사용가능
  - Chrome 개발자 도구를 사용하여 디버깅, 성능 개선 등의 작업을 수행할 수 있음
  - <https://github.com/ChromeDevTools/chrome-devtools-mcp>
- Toss payments MCP
  - Toss 결제 시스템에 연동하는 앱의 개발을 손쉽게 도와주는 MCP
  - <https://docs.tosspayments.com/guides/v2/get-started/llms-guide>
- PortOne MCP
  - AI를 이용해 포트원 결제시스템과의 연동을 손쉽게 할 수 있도록 도와주는 MCP
  - <https://developers.portone.io/opi/ko/integration/using-ai-tools?v=v2>

## 5. MCP

### ❖애플리케이션 개발시 자주 사용하는 MCP 서버(이어서)

- github
  - github 연동을 위한 MCP
  - <https://github.com/github/github-mcp-server>
- vercel
  - 프론트엔드 개발자를 위한 클라우드 플랫폼
  - 웹 애플리케이션의 배포와 호스팅에 특화된 서비스
  - <https://vercel.com/docs/mcp/vercel-mcp>
- Kubernetes
  - k8s 클러스터에 대한 연결, 관리 기능을 제공하는 MCP
  - kubeconfig 를 로딩하여 여러 클러스터에 접근할 수 있음
  - <https://github.com/Flux159/mcp-server-Kubernetes>
- sast-mcp
  - 오픈 소스 기반 보안 스캔 기능을 제공하는 mcp
  - <https://github.com/Skyrxin/sast-mcp-server>

## 5. MCP

### ❖ 애플리케이션 개발시 자주 사용하는 MCP 서버(이어서)

- shadcn MCP
  - shadcn ui란?
    - tailwind css 기반으로 만들어진 사용자 정의, 확장 및 구축이 가능한 예쁘게 디자인된 컴포넌트 집합
  - <https://ui.shadcn.com/docs/mcp>
- Webpixels MCP 서버
  - Bootstrap 5 컴포넌트를 검색, 조회, 조립할 수 있게 해주는 MCP 서버
  - <https://github.com/webpixels/mcp>
- Figma
  - Figma 디자인을 읽어와 코드로 추출하거나 프롬프트를 이용해 새로운 Figma 디자인을 생성할 수 있음
  - <https://www.figma.com/ko-kr/mcp-catalog/>

### ❖ 이밖에도 다양한 MCP 서버

- <https://glama.ai/mcp>
- <https://github.com/punkpeye/awesome-mcp-servers/blob/main/README-ko.md>
- <https://github.com/appcypher/awesome-mcp-servers>

## 5. MCP

### ❖주의 사항

- 가능하다면 외부 서버보다는 로컬 머신에서 실행하는 형태를 사용
  - Smithery AI와 같은 중개 서비스들도 있지만 기업 내부의 데이터가 외부로 유출되는 것이므로 가능하다면 로컬 머신에서 실행할 수 있는 형태를 고르는 편이 좋음
    - 로컬 머신에서 실행이 불가능한 것도 있음
  - 특히 http 원격 서버로 접속하는 방식인 경우 내부 정보가 유출될 수 있음
    - http 방식이라도 로컬 컴퓨터에서 서버를 실행하는 방법을 권장함
- github에 소스코드가 공개되었다면 소스코드 취약점 분석을 수행 후 사용할 것을 권장함
- 사용량이 많고 검증된 MCP 서버를 사용함
  - 검증되지 않은 것은 안정성의 문제, 보안 문제가 있을 수 있음
  - <https://glama.ai/mcp> 같은 사이트에서 호출/다운로드 수가 많은 것을 고르는 편이 안전함

# 5. MCP

## ❖설치 방법

- 자세한 설치방법은 해당 MCP 서버의 공식 문서를 참조하거나 AI에게 질의함
- 예시 : context7 mcp 사용
- 명령어로 설치하는 방법 : 전역 수준임. 프로젝트 수준의 추가 명령어 없음
  - `codex mcp add context7 -- npx -y @upstash/context7-mcp`
- 직접 설정파일에 추가하는 방법
  - `~/.codex/config.toml` 파일 (User 수준) 또는 `.codex/config.toml` (프로젝트 수준)에서 지정함
  - 프로젝트 수준의 mcp 설정을 권장함

```
[agents]
.....(생략)

[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]
```

## 5. MCP

### ❖ 권장되는 MCP 설치

```
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]
```

```
[mcp_servers.playwright]
command = "npx"
args = ["-y", "@playwright/mcp@latest"]
```

```
[mcp_servers.chrome-devtools]
command = "npx"
args = ["-y", "chrome-devtools-mcp@latest"]
```

```
[mcp_servers.supabase]
type = "http"
url =
"https://mcp.supabase.com/mcp?features=docs,account,database,debugging,development,functions,branching"
```

```
[mcp_servers.postgresql-mcp]
command = "npx"
args = ["-y", "@henkey/postgres-mcp-server"]
```

## 6. Codex Hook

### ❖ Hook 이란?

- 생명주기 과정 중 특정 이벤트(PreToolUse, PostToolUse, UserPromptSubmit, Stop 등)가 발생했을 때 미리 지정한 명령어나 MCP 도구를 자동 실행하는 기능
- "트리거 기반 자동화"

### ❖ 활용 예시

- 알림/피드백
  - 명령 실행이 끝나면 TTS(음성 합성)으로 “Command finished”를 읽어주기
  - Slack/Discord 같은 메신저에 실행 완료 알림 보내기
- 결과 처리
  - Codex가 생성한 Code Snippet을 자동으로 저장 파일에 추가하기
  - 결과물을 특정 포맷으로 변환(예: markdown -> HTML)
- 개발 워크플로우 통합
  - Git hook과 비슷하게 Codex 훅을 이용해 코드 생성 직후 자동으로 lint 기능이나 prettier 실행
  - CI/CD 시스템(Jenkins, Github Action)에 자동으로 트리거를 걸어 빌드/배포 실행

## 6. Codex Hook

### ❖ Hook에서 사용가능한 이벤트

- PreToolUse: 도구 호출 전에 실행 (차단 가능)
- PostToolUse: 도구 호출 완료 후 실행
- UserPromptSubmit: 사용자가 프롬프트를 제출할 때, Codex가 처리하기 직전 실행
- Stop: Codex가 응답을 완료할 때 실행
- SubagentStart : 서브에이전트가 작업을 시작할 때 실행
- SubagentStop: 서브에이전트 작업이 완료될 때 실행
- PreCompact: Codex가 압축 작업을 실행하려고 할 때 실행
- SessionStart: Codex가 새 세션을 시작하거나 기존 세션을 재개할 때 실행
- SessionEnd: Codex 세션이 종료될 때 실행

## 6. Codex Hook

❖ matcher : 어떤 대상에 대해서 Hook을 실행할지를 결정하는 필터 조건

- 이벤트명에 따라서 매칭하는 대상이 달라짐.

| 이벤트               | matcher가 비교하는 값 | 예                                  |  |
|-------------------|-----------------|------------------------------------|-------------------------------------------------------------------------------------|
| PreToolUse        | 도구 이름           | Bash , apply_patch , MCP 도구명       |                                                                                     |
| PostToolUse       | 도구 이름           | Bash , Edit , Write                |                                                                                     |
| PermissionRequest | 권한 요청 대상 도구 이름  | Bash , apply_patch , MCP 도구        |                                                                                     |
| SessionStart      | 세션 시작 방식        | startup , resume , clear , compact |                                                                                     |
| SessionEnd        | 종료 이유           | 현재는 other                          |                                                                                     |
| PreCompact        | compact 발생 방식   | manual , auto                      |                                                                                     |
| PostCompact       | compact 발생 방식   | manual , auto                      |                                                                                     |
| SubagentStart     | 서브에이전트 타입       | 예: explorer , worker 등             |                                                                                     |
| SubagentStop      | 서브에이전트 타입       | 예: explorer , worker 등             |                                                                                     |

## 6. Codex Hook

### ❖Codex Hook 예시1

- Stop 이벤트 시에 명령에 대한 응답이 완료되었음을 소리로 알려주는 훅
  - Mac은 TTS로 음성 울림
  - 윈도우는 알람 소리 울림

```
[features]
hooks = true
```

```
.....(중략)
```

```
[[hooks.Stop]]
```

```
[[hooks.Stop.hooks]]
```

```
type = "command"
```

```
command = "say ₩작업이 완료되었습니다.₩"
```

```
command_windows = "powershell.exe -NoProfile -Command ₩(New-Object Media.SoundPlayer
'C:₩₩Windows₩₩Media₩₩Alarm02.wav').PlaySync()₩"
```

```
timeout = 10
```

## 6. Codex Hook

### ❖Codex Hook 예시2

- 파일이 저장되면 Prettier가 작동되도록 함.₩
- settings.json 편집

.....(생략)

```
[[hooks.PostToolUse]]
matcher = "Write|Edit"
```

```
[[hooks.PostToolUse.hooks]]
type = "command"
command = "npx prettier --write . > /dev/null 2>&1"
command_windows = "npx prettier --write . > NUL 2>&1"
timeout = 60
```

## 6. Codex Hook

### ❖Codex Hook 예시 2(이어서)

- Prettier 설정 비활성화

- VSCode - 명령 팔레트 - 기본 설정: 사용자 설정 열기(JSON) 으로 이동후 Prettier 일시적으로 삭제

```
{
 (생략)
 "editor.formatOnSave": false,
 "editor.codeActionsOnSave": {
 "source.fixAll.eslint": "never",
 },
}
```

- src/test.json 파일 생성

```
{ "a": 100, "b":200 }
```

- 들여쓰기,띄어쓰기를 올바르게 하고 저장
- 프롬프트 실행
  - "a" 필드 값을 500으로 변경한다.
- 파일 수정과 동시에 들여쓰기 재조정 되는 것을 확인

# 7. Skill

## ❖Codex Skills

- Codex가 특정 작업을 일관된 방식으로 수행하도록 만드는 재사용 가능한 워크플로우 패키지
  - 하나의 작업(문서 변환, 템플릿 생성, 스타일 적용)을 위해 설계된 Skill.md와 추가 지침, 관련 스크립트의 집합
- 자동 호출 방식 : Codex가 컨텍스트를 확인하고 적절한 Skill을 선택해서 사용할 수 있음
- 작동 영역 : main context

## ❖Skill의 구성

- skill.md 파일(YAML 메타데이터 + markdown) + 추가적인 스크립트

## ❖Skill의 장점

- 토큰, 메모리 사용량 절감
  - claude code 세션이 시작될 때는 메타데이터만 로드하고 해당 skill이 필요할 때 전체 내용을 로드하기 때문에 토큰을 절약할 수 있고, 메모리 사용량을 줄일 수 있음
- 일관된 결과물
  - 정형화된 작업은 추가적인 스크립트가 수행하므로 일관된 결과를 기대할 수 있음
- 조합 가능
  - 여러 skill을 연결해 워크플로우 형태로 구성할 수 있음

# 7. Skill

## ❖ Skill 작성 예시

```

name: code-review
description: 코드 변경 사항을 검토하고 버그, 보안 문제, 테스트 누락을 찾을 때
사용한다.

```

# 코드 리뷰 절차

1. 변경된 파일을 확인한다.
2. 버그 가능성을 검토한다.
3. 보안 취약점을 확인한다.
4. 테스트 누락을 확인한다.
5. 심각도 순서대로 결과를 정리한다.

- md 파일에는 name, description 속성만 지정함

- 나머지 속성은 ./codex/skills/⟨my-skill⟩/agents/openai.yaml에 설정

```
my-project/
├── .agents/
│ └── skills/
│ ├── code-review/
│ │ └── SKILL.md
│ └── api-design/
│ └── SKILL.md
```

# 7. Skill

## ❖ Skill 작성 예시(이어서)

### ▪ openai.yaml 예시

```
interface:
 display_name: "Code Review"
 short_description: "변경된 코드를 검토합니다"
 icon_small: "./assets/icon.svg"
 icon_large: "./assets/logo.png"
 brand_color: "#3B82F6"
 default_prompt: "현재 변경된 코드를 리뷰해줘"
policy:
 allow_implicit_invocation: false
dependencies:
 tools:
 - type: "mcp"
 value: "github"
 description: "GitHub MCP server"
```

```
my-skill/
├── SKILL.md
└── agents/
 └── openai.yaml
```

### ▪ allow\_implicit\_invocation

- 이 값을 false 로 지정하면 작업 내용에 따라 Skill이 자동선택되지 않음
- \$code-review 와 같이 명시적으로 지정해야 함.

## 7. Skill

### ❖예시 : 백엔드 개발을 수행하는 사용자 정의 Skill

```

name: develop-backend
description: 실행계획의 BE Task를 구현·검증할 때 사용합니다. 해당 Task와 관련 문서, backend 코드를 확인한 뒤 최소 변경으로 완료 조건을 충족합니다.

```

사용자가 지정한 BE Task를 처리한다.

1. `docs/5-plan.md`에서 해당 Task와 선행 Task, 완료 조건을 읽고, 필요한 경우에만 `docs/1-prd.md`, `docs/2-user-scenario.md`, `docs/4-erd.md`, `backend/swagger.yaml`을 확인한다.
2. `backend`의 관련 코드와 기존 테스트·실행 명령을 먼저 확인한다. Task 범위 밖의 구조 변경이나 의존성 추가는 하지 않는다.
3. 완료 조건을 충족하는 최소 구현을 하고, 신뢰 경계·데이터 정합성·오류 응답은 명세를 따른다.
4. 앱 실행 시 로그가 남도록 기존 로거를 재사용한다. 없으면 기본 로깅으로 시작과 오류를 기록하며, 요청 본문·토큰·비밀번호·연결 문자열 등 민감정보는 기록하지 않는다.
5. 변경한 동작을 검증할 가장 작은 기존 또는 새 테스트를 실행하고, 실행 로그가 남는지도 확인한다. 완료 조건이 충족되지 않거나 테스트가 실패하면 테스트 출력과 관련 애플리케이션 로그를 확인해 원인을 고친 뒤 다시 검증한다.
6. 완료 조건이 충족될 때까지 3~5단계를 반복한다. 외부 서비스·환경 문제나 추가 권한 때문에 진행할 수 없으면 반복을 멈추고, 실행한 명령·관련 로그·원인·필요한 조치를 보고한다. 민감정보는 로그에 남기거나 보고하지 않는다.
7. 모든 완료 조건을 확인한 경우에만 `docs/5-plan.md`의 해당 체크박스에 체크표시를 한다.

## 7. Skill

### ❖ 기본 내장된 Codex Skill

- skill-creator : 새로운 skill을 만들거나 기본 skill을 개선할 때 사용
- skill-installer : OpenAI가 선별한 스킬 또는 다른 저장소의 Skill 을 설치할 때 사용
- plan : 복잡한 작업을 장기 실행 계획(ExecPlan)으로 구성

### ❖ skill-installer 로 설치할 수 있는 스킬들 - 언제든지 변경되거나 삭제될 수 있음

- <https://github.com/openai/skills>
  - deprecated 상태이나 사용 가능. 상당수가 plugin 으로 옮겨감
- <https://github.com/openai/plugins>

### ❖ 사용 방법

- Skill은 프롬프트로 수행하려는 작업 내용이 description에 명시된 경우와 매칭되면 자동으로 skill을 선택함
- \$skill-name 형태로 사용하여 명시적으로 skill 사용을 지정할 수 있음

## 8. 핵심 사용 팁

### ❖ Plan Mode 사용

- Plan mode란?
  - 작업 계획만 수립하며 실행하지는 않음.
  - 작업 계획을 계속해서 다듬어서 실행하는 것이 바람직함
- 사용 방법
  - /plan 명령어 사용
  - 채팅창에서 Plan Mode로 바꾸기 : Shift + Tab
- Plan Mode 대신에 markdown 형식으로 실행 계획을 작성하는 것이 바람직한 경우도 있음
  - plan mode의 단점
    - 직접 수정할 수 없음
    - 세션이 종료되면 plan이 사라짐
  - markdown 파일로 저장 --> 수정 후 @ 로 참조해서 최종 실행

## 8. 핵심 사용 팁

### ❖AGENTS.md 활용법

- Codex는 이 파일을 읽어서 프로젝트에 대한 정보를 파악하고 응답함
- AGENTS.md에 포함되면 좋을 것들
  - 주요 명령어 : 서버 구동 명령어, 빌드 명령어 등
  - 프로젝트 구조 및 아키텍처
    - 프로젝트의 디렉토리 구조
    - 핵심 구성 요소
  - 코드 스타일에 대한 지침
  - 테스트에 대한 지침
  - 반드시 지켜야 할 중요 지침 추가
    - 예시) 모든 대화는 한국어로
    - 예시) 이해하기 쉬운 용어로 설명
    - 예시) Over-engineering 금지
  - 최대한 200줄 이내로 간결하게 작성하는 것이 좋음

## 8. 핵심 사용 팁

### ❖ AGENTS.md 활용법 (이어서)

- AGENTS.md 파일은 계층적으로 작성하는 것이 바람직함
  - 예) /AGENTS.md, /frontend/AGENTS.md, /backend/AGENTS.md
  - 상위 AGENTS.md에는 포괄적인 지침, 하위 디렉토리의 AGENTS.md에 상세한 지침 작성
  - 이렇게 구성하면 Codex는 하위 디렉토리 관련 작업을 할 때만 하위 디렉토리의 AGENTS.md를 읽어들이므로 과도하게 컨텍스트를 낭비하지 않음
- ./AGENTS.override.md
  - 프로젝트 수준에서 개발자 자신만의 지침을 지정
  - 이 파일은 .gitignore에 등록해서 Git, Github 리포지토리에 반영되지 않도록 함

## 8. 핵심 사용 팁

### ❖ 정기적으로 /compact 수행

- /compact란?
  - 컨텍스트를 압축하여 요약하는 슬래시 명령어
  - Codex 의 컨텍스트 윈도우 크기는 1.05M (1.05백만 토큰)이며 90% 수준에 도달하면 자동으로 Auto Compact 가 수행되지만 직접 수행하는 것을 권장함
- 직접 /compact 수행을 권장하는 경우
  - 긴 분석/디버깅이 끝나고 새 작업 단계로 넘어갈 때
  - 많은 파일과 로그를 읽은 뒤 핵심 결론만 남기고 싶을 때
  - 서브에이전트 결과가 많이 쌓여 컨텍스트가 복잡해졌을 때
  - 이전 작업의 세부 내용이 이후 다음 작업에 거의 필요 없을 때
- 남아있는 Context Window는 /status 명령어로 확인하거나 /statusline 명령어로 설정하여 statusline을 통해서 확인

## 8. 핵심 사용 팁

### ❖Codex Plugin

- 관련된 Codex 확장 기능(mcp, skill, subagent, hook 등)을 하나의 배포 단위로 묶어서 한번의 명령어로 설치할 수 있도록 만든 패키지
- 공식 플러그인 저장소
  - <https://github.com/openai/plugins>
- 3rd-party 마켓플레이스
  - <https://github.com/cogine-ai/marketplace>
  - <https://github.com/topics/codex-marketplace?o=asc&s=forks>
  - <https://github.com/hashgraph-online/awesome-codex-plugins>

## 8. 핵심 사용 팁

### ❖ AGENTS.md 에 대한 fallback 파일명 지정

- 이미 다른 AI 코딩 도구로 개발하다가 Codex로 개발 도구를 변경했을 때 유용한 방법
- 예시 : Claude Code로 개발하다가 Codex CLI로 AI 코딩 도구를 변경하는 경우
  - 이미 CLAUDE.md 기본 지침이 존재함
  - 중복된 AGENTS.md를 해야 하나?
- .codex/config.toml 에 다음과 같은 내용 추가

```
[project]
project_doc_fallback_filenames = [
 "CLAUDE.md",
 "README.md"
]
```

## 9. 정리

### ❖ 도구 분류

- IDE형 (VSCode+AI 확장, Cursor/Windsurf) vs CLI형 (Claude Code, gemini-cli, Codex, qwen-code)로 구성.

### ❖ MCP 통합

- Settings에서 MCP 서버 등록(예: context7, sequential-thinking, playwright, shadcn, supabase, github, vercel, k8s)로 외부 데이터·도구 안전 연동.

### ❖ Codex

- /init, /status, /ps, /agents, /mcp, /skills 등 슬래시 명령어
- SubAgent, Hook(Pre/PostToolUse, Stop 등), Skill, MCP 로 자동화·품질 제어
- 실무 팁
  - Plan Mode(승인 게이트), 계층적 AGENTS.md
  - 정기적인 /compact로 컨텍스트 관리